

On this page

Requirements



This guide is just a quick reference of the requirements to install Pulse. For a comprehensive guide, please refer to the [official documentation](#).

Service Requirements

This service requires the following software to run.

Java Environment

OpenJDK SE Development Kit version 1.8, with the following vendors (with their latest hotfix versions):

- Azul Zulu [rpm](#) [zip](#) [tar.gz](#)
- Azul Platform Prime (formerly Zing)
- Amazon Corretto
- Oracle



Product tests are done with the **Azul Zulu JVM**.

The following versions are not supported, and are known to have problems:

- 1.8.0u242
- 1.8.0u121
- 1.8.0u60
- 1.8.0u45 and earlier

Non-listed releases are either not supported or haven't been put to extensive testing.

Setup Requirements

ZooKeeper

Pulse uses ZooKeeper for coordinating all of its components (Pulse Servers and Application Engines) in a datacenter. With ZooKeeper Pulse knows what components are available in a datacenter, is informed when one of them goes down or rejoins the datacenter, creates distributed locks, elects a leader, as well as maintains information about each component.

To setup Pulse with ZooKeeper, you should follow these steps:

1. Install a ZooKeeper ensemble and make sure it is working properly.
2. Configure `pulse.manager.cluster.dc.{dc}.zookeeper.nodes` with the nodes, or a load balancer (recommended). The `{dc}` parameter refers to the name of the datacenter the nodes belong to. It is defined in the `node.properties` file.
3. Configure the `node.properties` with the ZooKeeper namespace you wish to use. If you are preparing a coloured deployment, use the colour of the cluster of the nodes being configured. For example, you can use `pulseblue`.
4. Start Pulse. If the service is able to connect to ZooKeeper, a new namespace will be created automatically with the name you have specified in the previous step.



The current installation of Pulse has been tested with ZooKeeper 3.6.1.

Profile Store



All configurations referenced in this section are prefixed by `pulse.app.`

`{appSlug}.services.rte.workflow.plan.scheduledMetrics.profileStore` - this prefix is denoted in the examples below by `{PREFIX}`.

Pulse relies on a read-optimized NoSQL database system to store entity profile data (e.g. Customer-level aggregations), and be able to fetch in real-time without compromising latencies. The following systems are supported:

- Apache Cassandra 2.1 or higher - version 3.11.3 is the recommended version, and [DSE](#) is also supported

You can decide which store to use with the configuration `{PREFIX}.storage`. You must also specify the name of the table that will store the profile data. That is set by `{PREFIX}.table`.

Cassandra

When using Cassandra, Pulse will also automatically create the profiles table, considering the following:

1. A Cassandra cluster is available, and working properly. To connect to Cassandra, you must define the contact points in `{PREFIX}.cassandra.contactPoints` (for example, `cassandra-1,cassandra-2`).
2. A user has been created for Pulse (see how to create users in [this guide](#)). Configuring Pulse with the credentials to access the Cassandra cluster is done by setting `{PREFIX}.cassandra.user|password`. Pulse requires a user that has permissions to:
 - Create a keyspace
 - Create tables
 - Modify (Insert, Delete, Update, Truncate)
 - Select



Cassandra user credentials are stored in the `system_auth` keyspace, which is not replicated by default. This may make users unable to login if certain nodes are down. Read more about this in the [DataStax documentation](#).



To know more about the recommended Cassandra setup, refer to the [Pulse official documentation](#). To see which configurations can be set to connect to Cassandra see the [official configuration reference](#).

RDBMS

Pulse uses [PulseDB](#) for database access, which allows it to run on top of multiple database systems:

- Oracle 19.3 (or higher)
- PostgreSQL 10.1 (or higher) - driver version 42.2.18 should be used.
- SQL Server 2017 GA (or higher)
- H2 1.4.200 (or higher) - only recommended for testing purposes.



Feedzai validates the product primarily against **H2** and **PostgreSQL**.



Before selecting a database engine, make sure you understand how different database systems are [supported by our PulseDB](#).

Pulse relies on a relational database not only to store the events ingested and scored (also known as the "Event Storage"), but also to store metadata about the cluster and the applications that it serves. For that reason, before starting Pulse you must:

1. Provision a database with the engine of your choice.
2. Create a new user for Pulse.
3. Create a new schema for Pulse. If you are preparing a coloured deployment (i.e. to support Blue-Green deployments), you must create two schemas. For example, `pulseblue` and `pulsegreen`.
4. Update the Pulse configurations with the JDBC connection string, the schema, and the credentials for the user you just created.

The database settings must be configured for the following prefixes:

- Pulse Server Metadata - `pulse.global.database`
- Pulse AppEngine Metadata - `pulse.app.{appSlug}.database`
- AppEngine Event Storage - `pulse.app.{appSlug}.services.modules.rte.workflowmanager.eventStorage.database`

To connect to the database you need to provide the `jdbc`, `schema`, `username` and `password` settings.



To see which configurations can be set to connect to a database see the [official configuration reference](#).

RabbitMQ

RabbitMQ is the messaging platform Pulse uses to communicate between machines and processes within the same datacenter.

The following RabbitMQ and Erlang versions are supported:

- RabbitMQ - 3.8.23 with Erlang - 23.3.4+
- RabbitMQ - 3.9.8 with Erlang - 23.3.4+
- RabbitMQ - 3.13.7 with Erlang - 26.2.5+
- RabbitMQ - 4.2.3 with Erlang - 27.3.4+

Before starting Pulse you must configure the messaging cluster:

1. Provision a RabbitMQ cluster and ensure there Pulse is able to establish a connection.
2. Create a new vhost for Pulse. If you are preparing a coloured deployment (i.e. to support Blue-Green deployments), you must create two vhosts. For example, `pulseblue` and `pulsegreen`.

```
rabbitmqctl rabbitmqctl add vhost pulseblue
rabbitmqctl rabbitmqctl add vhost pulsegreen
```

Copy

1. Create a new user for Pulse.

```
rabbitmqctl add_user pulse <password>
```

Copy

1. Update the vhost policies to grant all permissions to the Pulse user.

```
# set write and read permissions on every resource
rabbitmqctl set_permissions pulse -p pulseblue ".*" ".*" ".*"
rabbitmqctl set_permissions pulse -p pulsegreen ".*" ".*" ".*"
```

Copy



On single-node and testing environments, you can opt by using ActiveMQ as a local, embedded message broker. Nonetheless, ActiveMQ is not recommended even for standalone Production setups (e.g., using a standalone Pulse for model training activities).

No-Downtime Deployments🔗

Pulse uses a **Blue-Green** deployment strategy to implement no-downtime deployments. In order to do that, the architecture of the Feedzai platform needs to be prepared for the deployment switching.

Blue-green deployment uses two almost identical clusters in the following way:

1. At the start, one of the clusters is in use. Let's call it the blue environment.
2. The other one is on standby, in the final testing phase of a development project. Let's call this one the green environment.
3. When the new software version has been validated, the system gets redirected to the green deployment, and the blue deployment becomes the stand-by environment. It is available in case of a necessary rollback.
4. When the new version is verified in production, the blue deployment can be upgraded, making it identical to the green one. While still on stand-by, the blue environment is ready to receive a new development project.

To support the segregation between the blue and the green clusters, it is necessary to do the following:

- Create coloured vhosts in RabbitMQ (the messages from each cluster are isolated).
- Create coloured namespaces in ZooKeeper (each cluster then has its own cluster). Pulse will actually create it automatically if it doesn't exist, but it still needs to be configured differently for the two clusters.

When starting a new deployment, the cluster being deployed must be configured with the opposite colour.

```
/opt/feedzai/pulse/config/node.properties
node.cluster=pulsegreen
Copy
/opt/feedzai/pulse/config/config.properties
configuration.clusterTablesPrefix=green
Copy
/opt/feedzai/pulse/config/pulseinit.properties
pulse.manager.cluster.dc.dcl.messaging.vhost=pulsegreen
Copy
```

While the database configuration remains the same, the metadata tables are prefixed automatically with the value set by the `clusterTablesPrefix` property.



The colour of the deployment should be a template in the configuration files, so that it can be updated dynamically when a new service instance is provisioned. This is particularly relevant in setups where immutable VM images are created and the same one is reused across the multiple environments (usually the case on Cloud-based setups).

RabbitMQ Federation🔗

During a deployment, the active cluster continues to process events and synchronising state between the cluster nodes. However, if that state is not shared with new cluster, scoring will be impaired after switching the traffic to it. For that reason, it is important to **federate the event replicas between between the two clusters**. This is done with the [RabbitMQ Federation](#) plugin.

To connect the green cluster to receive updates from the blue one, you must create a federation policy, and then apply it to the the queues

```
rabbitmqctl set_parameter [-p vhost] federation-upstream blue-green '{"uri": ["amqp://<user>:<pass>@<node 1>/pulseblue", "amqp://<user>:<pass>@<node 2>/pulseblue", ...]}'

rabbitmqctl set_policy [-p vhost] federate-blue-green 'global-fdz-metadata-sync|global-fdz-apps-
service|global-fdz-tenant-sync|global-fdz-tenant-group-sync|global-fdz-sync-topic-.*'
'{"federation-upstream": "blue-green"}' --apply-to exchanges
```

Copy



caution

When doing a Blue-Green deployment, only the topics above should be federated, otherwise there could be unwanted control messages being propagated between the two clusters (e.g. stopping the application).

Data Science Studio

Even though Pulse is commonly referred to as an Risk Engine and used for monitoring fraud in real-time, it is in fact a Data Science platform where the data analytics tooling can be integrated with a runtime environment.

The following are the requirements for setting up Pulse for a Data Science environment:

- A database for metadata storage, ideally a Relational one, such as PostgreSQL
- 1 machine for Pulse to execute on (a single replica)
- Coordination processes, which can be dockerized and/or executing in the same machine as Pulse:
 - 1 RabbitMQ process
 - 1 ZooKeeper process
- A Spark cluster for distributed job execution. The recommended approach is to run Spark on YARN (Cloudera, AWS EMR, Azure HDInsight, Google Dataproc), which requires a compliant Hadoop File System to exist in the cluster.



warning

Feedzai **does not recommend** the use of embedded coordination processes (i.e., using **embedded ZooKeeper** and **ActiveMQ**), as well as databases that are not production-ready, like **H2**.

For more information about the Data Science setup, refer to the [official documentation](#).

Integration with JupyterLab

Pulse supports integration with [JupyterLab](#).

- To enable Setup of JupyterLab:
 1. Docker must be installed on the Pulse machine.
 2. [Feedzai Jupyterlab Docker Image](#) must exist on Pulse machine.
- If the Docker software is not available on the Pulse machine, so the flag **"pulse_jupyterlab_setup_docker"** must be set to false, so that Pulse doesn't configure Jupyterlab erroneously.